Experiment 6

Word Count Application using Hadoop MapReduce

Subject: Data Science Lab | Environment: Hadoop 3.4.1 + Java 17 | Mapped CO(s): CO2, CO3

1. Aim

To implement a Word Count application using the Hadoop MapReduce framework and analyze the frequency of words in a given text dataset.

2. Key Concepts

MapReduce

Programming model for processing large datasets in parallel. A job is split into a Map phase and a Reduce phase.

Mapper

Reads one input record (a line) at a time and emits intermediate (key, value) pairs. For Word Count it emits (word, 1).

Shuffle and Sort

Framework step between Map and Reduce. Groups all values with the same key and sends them to one Reducer, sorted by key.

Reducer

Receives (key, list of values) and aggregates them. For Word Count it sums the 1s to get (word, total).

Combiner

Optional mini-reducer that runs on Mapper output locally to reduce data sent over the network. Word Count reuses the Reducer.

InputSplit / RecordReader

The input file is split into chunks; TextInputFormat's RecordReader gives the Mapper (byte offset, line) pairs.

Driver

The main() class that configures the Job: Mapper, Combiner, Reducer, output types, input path, and output path.

Writable types

Hadoop's serializable types: Text for strings, IntWritable for ints, LongWritable for offsets.

3. Data Flow

$$(offset,\ line) \rightarrow (word,\ 1) \rightarrow (word,\ [1, 1,\ \ldots]) \rightarrow (word,\ \sum 1)$$

Map -> Shuffle/Sort -> Reduce

Count of a word w over all lines:

$$count(w) = \sum_{i=1}^{N} \mathbb{1}[token_i = w]$$

Phase	Input	Output
Map	(LongWritable, Text line)	(Text word, IntWritable 1)
Combine	(word, [1,1,...]) per mapper	(word, partial sum)
Shuffle/Sort	All mapper outputs	(word, [values]) grouped
Reduce	(word, [values])	(word, total count)

4. Algorithm

- 1. Start HDFS (NameNode + DataNode) and upload input.txt to /user/lab6/input.
- 2. Mapper: lowercase each line, remove punctuation, split into tokens, emit (token, 1).
- 3. Combiner: locally sum counts for each word on the map side.
- 4. Shuffle/Sort: framework groups pairs by word.
- 5. Reducer: sum all counts for a word and emit (word, total).
- 6. Driver: configure the Job, set input/output paths, submit and wait.
- 7. Compile with javac using `hadoop classpath`, package into wordcount.jar.
- 8. Run: hadoop jar wordcount.jar WordCount <input> <output>.
- 9. Read result: hdfs dfs -cat <output>/part-r-00000.

5. Commands

Command	Purpose
hdfs dfs -mkdir -p /user/lab6/input	Create input directory
hdfs dfs -put input.txt /user/lab6/input	Upload input
javac -classpath $(hadoop classpath) ...	Compile WordCount.java
jar cf wordcount.jar -C build .	Package classes into JAR
hadoop jar wordcount.jar WordCount in out	Run the job
hadoop jar ... -D wordcount.skip.stopwords=true	Run with stopword filter
hdfs dfs -cat out/part-r-00000	View output
hdfs dfs -rm -r out	Delete old output (must not exist)

6. Sample Output (input.txt)

Word	Count	Word	Count
a	1	hadoop	2
big	2	is	1
data	2	mapreduce	2
efficiently	1	processes	1
framework	1	uses	1

Total words = 14, unique words = 10. Counters: Map output records = 14, Combine output records = 10, Reduce output records = 10.

7. Exercise Answers

Exercise 1: Modify input and observe output

Two lines were added (HDFS stores big data in blocks. / Hadoop is open source and Hadoop is scalable.). Total words grew from 14 to 28 and unique words from 10 to 18. hadoop: 2 -> 4, is: 1 -> 3, big and data: 2 -> 3.

Exercise 2: Ignore common words

The Mapper checks a STOPWORDS set (a, an, the, is, and, in, of, ...) when the job property wordcount.skip.stopwords=true is passed with -D. Words a, and, in, is disappeared from the output; Map output records dropped from 28 to 22.

Exercise 3: Run on another text file

another.txt (4 lines about data science and Hadoop) produced 39 words, 29 unique. Top words: data (4), the (4), and (3), distributed (2), science (2). The same JAR works on any text input.

Exercise 4: Compare before vs after

Joining the two outputs on word shows unchanged words (mapreduce, framework, uses...), increased words (hadoop +2, is +2, big +1, data +1), and new words (hdfs, blocks, open, source, scalable, stores, and, in).

Exercise 5: Role of Mapper and Reducer

- Mapper: splits each line into words and emits (word, 1). Runs in parallel on every input split; it does no counting across lines.
- Shuffle/Sort: groups identical words together and delivers them to the same Reducer.
- Reducer: adds up all the 1s for a word to produce its final frequency.
- Combiner: same logic as the Reducer applied locally to cut network traffic.

8. Viva Q&A

Q: What is MapReduce?

A: A programming model that processes data in parallel using a Map phase and a Reduce phase.

Q: What does the Mapper emit in Word Count?

A: (word, 1) for every word in the line.

Q: What does the Reducer do?

A: Sums the list of counts for each word and writes (word, total).

Q: What is a Combiner?

A: A local reducer on the map side that reduces data transfer. It must be associative and commutative.

Q: What is Shuffle and Sort?

A: The framework phase that transfers Mapper output to Reducers, grouped and sorted by key.

Q: Why Text and IntWritable?

A: Hadoop needs serializable Writable types to move data between nodes.

Q: Input format used?

A: TextInputFormat: key = byte offset (LongWritable), value = line (Text).

Q: Why must the output directory not exist?

A: Hadoop refuses to overwrite output to avoid accidentally losing results.

Q: What is part-r-00000?

A: Output file written by Reducer number 0. One file per reducer.

Q: What is _SUCCESS?

A: Empty marker file written when the job finishes successfully.

Q: How many mappers run?

A: Roughly one per input split (block). A small file gives one mapper.

Q: How did you skip stopwords?

A: Passed -D wordcount.skip.stopwords=true; the Mapper reads it from the Configuration in setup().

Q: What is ToolRunner?

A: Helper that parses generic options like -D before calling run(), so properties reach the job Configuration.

Q: Applications of Word Count?

A: Search indexing, log analysis, text mining, trending topics, document similarity.

9. One-Line Definitions

Mapper

Converts input records into (key, value) pairs

Reducer

Aggregates all values for a key

Combiner

Map-side local aggregation

Partitioner

Decides which Reducer gets a key

Shuffle

Moves map output to reducers

Job

One complete MapReduce program run

InputSplit

Chunk of input handled by one Mapper

Writable

Hadoop serialization interface

Driver

Class that configures and submits the job

LocalJobRunner

Runs MapReduce in a single JVM for testing